

2026 · 犀牛鸟开源人才培养

腾讯开源 · 开源课题实战

项目申请书

tRPC · 多租户与节点部署架构设计

牛晨勋

中南大学 · 数据科学与大数据技术 · 2023 级

 GITHUB  STELQUIS  CNB  Stelquis  邮箱 17837343195@163.COM

"Always a learner, always learning from the best."

项目申请书

项目：tRPC · 多租户与节点部署 申请人：牛晨勋 学校：中南大学 · 数据科学与大数据技术 · 2023 级

邮箱：17837343195@163.com / 3420761503@qq.com

► 课题背景

◆ tRPC-Agent 是腾讯开源的生产级 AI Agent 开发框架，框架核心围绕 LlmAgent 的推理 → 工具调用 → 回填 → 再推理 交替循环设计，内置 Filter 治理链、多后端 Session/Memory 抽象、Skill 技能体系、GraphAgent 图编排引擎以及 A2A / AG-UI 跨语言通信协议。

框架目前提供两个稳定语言版本，版本定位互补：

tRPC-Agent-Python

SDK 完整 企业微信 Filter 治理 A2A AG-UI

SDK 能力完整，内建 OpenClaw 运行时（含企业微信通道、Telegram 通道、CLI 交互），多后端 Session 服务（InMemory / SQL / Redis），多后端 Memory 服务，Filter 治理体系和 A2A/AG-UI 服务端，适合快速开发和 Agent 逻辑编排。

tRPC-Agent-Go

架构对等 九种后端 Guardrail 高吞吐

架构对等，Session 后端覆盖更广（SQLite / MySQL / PostgreSQL / Redis / MongoDB / ClickHouse 等九种），Memory 后端同样丰富，内建 Guardrail 插件体系和 OpenClaw Gateway 实现，goroutine 并发模型天然适合高吞吐网络层。

◆ 两个版本通过 A2A 协议 可跨语言互通，为多语言混合部署提供了协议基础。

本实战任务以 tRPC-Agent 框架为基础，在其现有能力之上构建 多租户隔离 与 分布式部署 层，可拆解为五个核心子问题：

◆ 子问题一：租户模型与配置管理

设计 Tenant 数据模型，核心字段涵盖 tenant_id、app_config、model_config、tool_whitelist、channel_binding、data_backend、audit_policy。每个租户拥有独立的配置空间，模型配置、工具权限、数据后端选择和审计策略运行时互不干扰。

◆ 子问题二：租户感知路由与数据分片

在 Gateway 和 Storage 层引入 tenant_id 路由机制，使不同租户的消息、会话、记忆和知识库能路由到各自指定的后端实例。每个租户可独立选择数据后端类型（InMemory Redis SQL 向量库 对象存储）。

◆ 子问题三: Gateway ↔ Worker 分离与水平扩展

将 Agent 执行从网关进程中解耦为独立的 Worker 池。Worker 为无状态设计，不持有会话状态，所有状态写入共享后端（Redis / SQL），实现按流量水平扩缩容，不依赖 Sticky Session。

◆ 子问题四: IM 通道标准化接入与租户绑定

设计统一的 Channel Adapter 接口，将企业微信、微信公众号等 IM 消息标准化为 Agent 输入。通过用户身份映射机制将 IM 用户的 open_id 绑定到具体 tenant_id，处理消息验签、去重和群聊/单聊 session_id 生成。

◆ 子问题五: 管理面 (Admin API) 与运维

提供租户 CRUD、配置变更、策略调整、灰度发布与配置回滚的标准化管理接口。配套

OpenTelemetry

全链路 Trace

审计日志

，支持 Docker Compose 最小部署与 Kubernetes 生产部署方案。

► 技术方案

前述五个子问题可以归结为一条极简的主线：两台机器、两个租户、两条消息同时到达，系统能正确处理。展开来说就是：两个不同租户的用户，各自发一条消息到同一个 Agent 服务，它们能不能互不干扰地得到各自的回复？这就是本方案要回答的核心问题。

"互不干扰"意味着两条线：配置不串（A 租户用 DeepSeek、B 租户用 GPT-4，各自配置各自生效）和数据不混（A 的会话和记忆不会出现在 B 的查询结果里）。"同一个 Agent 服务"意味着所有租户共享同一套计算节点和接入层，不能为每个租户单独部署一套。

围绕"配置不串、数据不混、水平扩展"三条要求，方案从三个层面递进设计：

① 数据层: 配置即命名空间

多租户的根本是按 tenant_id 分片。一张 tenants 表存所有租户配置（model_config、tool_whitelist、channel_binding、data_backend、audit_policy），每个配置项以 tenant_id 为前缀。框架已有的 SessionService（Python 的 Sql/RedisSessionService、Go 的 session/redis）只需在键空间上追加 tenant_id 前缀即可实现数据隔离。流程是：Gateway 收到请求 → 提取 tenant_id → Worker 根据 tenant_id 加载对应 config → 读写对应键空间的数据。API Key 只存引用 ID，详细配置以 JSON 字段存于 tenants 表，简单可扩展，不做物理隔离。

不做物理隔离是权衡后的选择——物理隔离需要为每个租户独立部署一套后端实例，管理成本随租户数线性增长，在多租户共享计算节点的架构下反而引入了更大的运维复杂度。逻辑隔离加 tenant_id 前缀覆盖了当前阶段的核心诉求：配置不串、数据不混。同时也保留了升级路径——需要时只需改动 StorageRouter 的路由策略即可切换到独立后端，不影响上层 Agent 逻辑。

② 计算层：无状态即水平扩展

Worker 要能自由扩缩，前提是不持有任何会话状态。这一点框架天然支持：Python 和 Go 的 SessionService 实现都将 event、state 写入了共享后端（Redis / SQL），Worker 只需 get_session() 就能恢复现场。Gateway 按 tenant_id 哈希分配 Worker，同一 session 尽量保持亲和（亲和信息存 Redis），但不依赖 Sticky Session——Worker 宕机后亲和信息丢失，Gateway 自动重分配，会话数据在共享后端不受影响。

亲和信息带 TTL，核心作用是减少 Session 在不同 Worker 间迁移的频率，而非保证固定绑定。TTL 超时或 Worker 宕机后，Gateway 重新哈希分配，新 Worker 从共享后端拉取完整会话状态继续执行。这里的关键在于“中断不丢失”而非“不断线”——IM 通道天然能接受异步回复，短暂的重分配延迟在用户体验上可接受。探活机制通过健康检查接口实现，Python 版复用 OpenClaw 通道层，Go 版使用 gRPC health probe。

③ 接入层：IM 通道即标准化适配器

不同 IM 通道有各自的 webhook 格式和消息协议，但进入 Worker 后都走同一套 Agent 逻辑。设计 Channel Adapter 接口，将企业微信回调、微信公众号事件等标准化为 InboundMessage（channel / from / to / thread / text），再转为 Runner.run_async() 输入。企业微信通道复用 Python OpenClaw 已有的 channels/_wecom.py（含流式回复），微信公众号为本次新增。消息去重通过 channel + message_id 做 SETNX 5 分钟 TTL 实现。session_id 按 sha256(channel + tenant_id + user_open_id) 生成，群聊追加 group_id。

Adapter 的职责止于消息格式转换——验签、去重、身份映射在 Adapter 层完成，之后的业务逻辑全部交给 Worker。这样做的好处是每接入一个新通道只需新增一个 Adapter 实现，Worker 和下游逻辑完全不动。session_id 加入 tenant_id 是为了跨租户场景下同一用户在两个租户的会话天然隔离；群聊追加 group_id 是为了区分群内 @机器人和单聊的消息流——两者在当前 Agent 事件模型中需要不同的 session 生命周期管理。

治理与运维

Filter 治理

- 工具白名单：Python 版用 @register_tool_filter 注册租户级 Filter（复用 Issue #90 SafetyFilter 的决策链）
- Go 版通过 Guardrail 插件的 approval 机制实现危险工具二次确认
- 敏感信息脱敏：12 种正则模式，IM token / API Key / 密码在日志中以 *** 呈现
- 审计日志字段：tenant_id、channel、user_id、session_id、agent_name、tool_name、decision、latency、error_type、cost、trace_id

监控与故障恢复

- OpenTelemetry trace 串起 IM → Gateway → Runner → Tool → Session/Memory → IM 全链路
- Worker 宕机：探活超时 → 重分配 session，会话状态在共享后端不丢失
- 数据库不可用：Memory 降级为本地 InMemory 缓存，恢复后异步同步
- 模型超时：设 max_retries，返回错误信息给用户并记录审计

- 数据迁移：双写 → 一致性校验（可复用 Issue #89 DiffEngine）→ 切换读流量 → 下线旧后端

1 个 Gateway

统一入口
按 tenant_id 路由

N 个 Worker

无状态执行
Docker Compose 一键拉起

K8s

生产部署
HPA 自动扩缩

三层合在一起就是：数据层按 tenant_id 分片管好"配置不串、数据不混"，计算层通过无状态 Worker 管好水平扩展，接入层用 Channel Adapter 管好多通道统一接入。这是本课题的整体方案框架。以下以周为单位规划交付节奏：

► 时间规划

实战周期：2026 年 8 月 1 日 ~ 9 月 10 日。组委会将分别在 8 月初、8 月底联系项目导师获取入围学生的实战情况与评价，以下以"周"为单位安排：

第 1 周

● 调研与设计 · 数据层基建

设计 Tenant 数据模型 (tenants / agent_apps / channel_bindings / audit_logs)，完成 StorageRouter 抽象接口，评估现有 SessionService 各后端 (InMemory / SQL / Redis) 的键空间改造方案

■ Tenant 模型设计文档 + StorageRouter 接口代码 + 键空间改造方案

第 2 周

⚙️ 核心开发 · 计算层解耦

实现 StorageRouter 多后端适配 (Redis / SQL)，实现 Agent Gateway 的 tenant_id 路由与 Worker 池无状态调度，接入 OpenTelemetry 全链路 trace

■ 多租户存储层可运行 + Gateway 路由 + 单元测试

第 3 周

✳️ IM 集成 · 接入层标准化

实现 企业微信 (复用 OpenClaw channels/_wecom.py) + 微信公众号/客服 的 Channel Adapter，完成回调验签、SETNX 去重、open_id → tenant_id 映射、群聊/单聊 session_id 生成、平台限制处理

■ 双通道 IM 适配器可运行 (Telegram 预留接口)

第 4 周

● 打磨交付 · 治理 + 部署 + PR

实现 Filter 治理 (租户级工具白名单 / 敏感信息脱敏 / 危险工具二次确认)、Admin API (租户 CRUD / 配置回滚)、Docker Compose 一键部署方案、提交 PR

■ 完整示例 + 部署文档 + PR 提交

★ 期望获得的成长与收益

▣ 架构设计能力

通过多租户 Gateway ↔ Worker 解耦、StorageRouter 多后端路由和 IM Channel Adapter 标准化接入的实战，理解从"单进程 Agent"到"分布式多租户服务"的架构演进路径。

■ 系统化思维

在数据一致性取舍 (强一致 vs 最终一致)、数据迁移策略 (双写切换)、租户隔离强度 (逻辑隔离 vs 物理隔离) 等现实约束下做工程决策，提升从"功能实现"到"系统设计"的思维层次。

★ 开源协作经验

深度参与 tRPC 社区的 PR 评审流程，学习与官方维护者的协作方式，将多租户方案沉淀到框架主干并长期维护。

🔗 期望熟悉的技术与模块

tRPC-Agent SDK

A2A 协议

AG-UI 协议

Filter 治理链

Session 多后端

Memory 多后端

StorageRouter

OpenTelemetry

Docker Compose

Kubernetes

企业微信 API

微信公众号 API

► 开源经历

在正式申请本课题前，我围绕 tRPC-Agent-Python 框架完成了四个阶段的开源实践，覆盖从基础到综合交付的完整链路：

三 阶段一：Issue #89 — Session/Memory 多后端回放一致性测试

Session

Memory

InMemory

SQLite

Redis

构建 Replay Harness 驱动 InMemory/SQLite/Redis 三种后端，10 个 Replay Case 覆盖 event order / state / memory / summary 四维度，DiffEngine 字段级比较。验收：29 项 replay 测试通过（0.93s），与已有 337 项 session 测试无冲突、零影响。

🛡️ 阶段二：Issue #90 — Tool Script Safety Guard

Filter

AST

OTel

策略引擎

可插拔 SafetyFilter 前置扫描 7 类风险，Python 脚本 AST 分析 + Bash 正则匹配，输出 allow / deny / needs_human_review 三级决策。验收：60 项测试通过，500 行脚本扫描 9.13ms。

📦 阶段三：Issue #92 — Skills + 沙箱 + 数据库代码评审 Agent

Skill

CodeExecutor

GraphAgent

A2A

AG-UI

构建自动化代码评审 Agent，整合 Skills 体系、Container/E2B 沙箱、SQLite 存储、GraphAgent 图编排和 Filter 治理。交付 5 类风险规则、6 张 DB 表、7 节点图编排管道和 8 条测试 diff。

△ 阶段四：ReviewMind — 综合 Demo（基于 #92 演进）

Eval

A2A Server

AG-UI Server

评测集

OpenTelemetry

在 Issue #92 骨架之上扩展为完整可服务的代码评审系统：新增 A2A/AG-UI 双协议服务端、RAG 知识库检索、长期记忆（SqliteMemoryService + 30 天 TTL）、ProgressReporter 流式进度推送、全套 Eval Set 评测和隐藏样本盲测。验收：8 条公开 fixture 全部通过，pytest 测试套件通过，共 37 个 Python 文件。

从浅入深，四个阶段依次覆盖了 Session/Memory 多后端、Filter 治理链、Skill + 沙箱 + 图编排、A2A/AG-UI 服务化与评测体系——这些正是本课题（多租户存储路由、租户级 Filter 治理、Gateway ↔ Worker 解耦）的直接前置经验。

► 个人陈述

四个阶段的实践经历让我清楚地意识到一件事：Agent 框架的工程化落地和写一个能跑的 Agent 脚本是完全不同的两件事。脚本只需要调通 API、拿到返回就够了；但一个工程化的 Agent 服务需要考虑模型超时怎么办、工具执行失败怎么恢复、多个会话之间的状态怎么隔离、不同业务方共享同一套基础设施时配置怎么不串、数据怎么不混。

这些问题的答案不在 LLM 的返回里，而在工程架构的设计里。而“多租户与节点部署”这个课题恰好站在这个交汇点上——tRPC-Agent 已经解决了 Agent“能跑起来”的问题，但多业务方共享时的配置隔离、数据分片、无状态 Worker 扩缩、IM 通道统一接入、治理监控体系，这些是企业落地的最后一道门槛。这也是我认为这个课题最有价值的地方：它的产出不会只停留在仓库里，而是能直接推动框架向企业级服务迈进。

双版本的存在提供了更大的设计空间。Python 版内建的 OpenClaw 通道层、Filter 治理链和多后端 Session 抽象是可复用的基础设施；Go 版的 Gateway 架构和 goroutine 并发模型适合做高吞吐路由。A2A 协议让两者可以协同工作而不是二选一。

从 Session/Memory 多后端测试（#89）到 SafetyFilter 治理链（#90），从 Skill 沙箱执行（#92）到完整的服务化系统（ReviewMind），四次实践恰好覆盖了本课题涉及的四个核心维度：存储抽象、治理策略、执行环境和接入协议。这不是从头学起，而是将这些已有能力组合并扩展到多租户场景——这个课题就是我在这条路上的自然延伸。

综上，我已在学习实践中验证了对 Session/Memory 多后端、Filter 治理链、GraphAgent 图编排、A2A/AG-UI 服务化和 Evaluation 评测体系的掌握。本课题是这些能力的横向延伸而非从头学起，我有信心按时交付高质量成果并持续为 tRPC 社区做出贡献。

申请项目：tRPC（第一志愿）· Tencent YOLO-Master（第二志愿）· WeKnora（第三志愿）